C# Entity Framework Cheatsheet

2026-03-19

Table of Contents

1. Ziel . . . 1
2. Architektur . . . 2
3. Entity Template . . . 3
4. Fluent API . . . 4
   4.1. Configuration Template . . . 4
   4.2. DbContext Template . . . 4
   4.3. Migration Ablauf . . . 5
5. Repository + UnitOfWork Muster . . . 6
   5.1. Contracts in Core . . . 6
   5.2. UnitOfWork in Persistence . . . 6
   5.3. Query-Regeln für gute Read-Methoden . . . 6
6. ReadModels / Overview-Objekte . . . 7
   6.1. Typisches Query-Template . . . 7
7. API Design Muster . . . 8
   7.1. DTO statt Entity direkt verwenden . . . 8
   7.2. Endpoint-Satz (Minimal API) . . . 8
   7.3. API Fehlerbehandlung . . . 8
8. Validierung (FluentValidation) . . . 9
9. Import-Pipeline Muster . . . 10
   9.1. Stabiler Ablauf . . . 10
   9.2. Import Template . . . 10

Chapter 1. Ziel

Dieser Cheatsheet ist bewusst allgemein gehalten. Er hilft dir, typische Schul-/Uebungsaufgaben mit einer mehrschichtigen C#-Architektur schnell und sauber umzusetzen.

Er deckt ab:

- Projektstruktur (Core, Persistence, API, Import/Tools)
- EF Core mit Fluent API
- Repository + UnitOfWork
- REST Endpoints + Validierung
- Reporting/Overview Queries
- Import aus Dateien

Chapter 2. Architektur

```
Core
  - Entities, Enums, Value Objects
  - Interfaces (Repositories, UnitOfWork)
  - QueryResult/ReadModels

Persistence
  - DbContext
  - Entity Configurations (Fluent API)
  - Repository Implementierungen
  - UnitOfWork Implementierung

WebAPI
  - DTOs
  - Endpoint Mapping
  - Validatoren
  - Dependency Injection Registrierung

Import/Console
  - Datei einlesen
  - Daten transformieren
  - Über Repositories speichern
```

Chapter 3. Entity Template

```
public class EntityName : EntityObject
{
    public string RequiredText { get; set; } = string.Empty;
    public DateTime CreatedAt { get; set; }

    public int ParentId { get; set; }
    public ParentEntity Parent { get; set; } = null!;
}
```

Chapter 4. Fluent API

Nutze Fluent API fuer DB-Regeln.

4.1. Configuration Template

```
public class EntityNameConfiguration : IEntityTypeConfiguration<EntityName>
{
    public void Configure(EntityTypeBuilder<EntityName> builder)
    {
        builder.HasKey(x => x.Id);

        builder.Property(x => x.RequiredText)
            .IsRequired()
            .HasMaxLength(100);

        builder.HasOne(x => x.Parent)
            .WithMany(p => p.Children)
            .HasForeignKey(x => x.ParentId)
            .OnDelete(DeleteBehavior.Restrict);

        builder.HasIndex(x => x.RequiredText);
    }
}
```

4.2. DbContext Template

```
public class ApplicationDbContext : DbContext
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options) :
base(options) { }

    public DbSet<EntityName> EntityNames { get; set; } = null!;
    public DbSet<ParentEntity> ParentEntities { get; set; } = null!;

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.ApplyConfigurationsFromAssembly(typeof(ApplicationDbContext
).Assembly);
    }
}
```

4.3. Migration Ablauf

Entweder über die Konsole oder über: Tools > Entity Framework Core > Add Migration / Update Database

```
dotnet ef migrations add InitialCreate --project .\Persistence\Persistence.csproj
--startup-project .\WebAPI\WebAPI.csproj

dotnet ef database update --project .\Persistence\Persistence.csproj --startup-project
.\WebAPI\WebAPI.csproj
```

Chapter 5. Repository + UnitOfWork Muster

5.1. Contracts in Core

```
public interface IEntityRepository : IGenericRepository<EntityName>
{
    Task<IList<EntityOverview>> GetOverviewAsync(string? filter, bool? onlyActive);
}

public interface IUnitOfWork : IBaseUnitOfWork
{
    IEntityRepository Entities { get; }
    IAnotherRepository Others { get; }
}
```

5.2. UnitOfWork in Persistence

```
public class UnitOfWork : BaseUnitOfWork, IUnitOfWork
{
    public UnitOfWork(ApplicationDbContext dbContext) : base(dbContext)
    {
        Entities = new EntityRepository(dbContext);
        Others = new AnotherRepository(dbContext);
    }

    public IEntityRepository Entities { get; }
    public IAnotherRepository Others { get; }
}
```

5.3. Query-Regeln für gute Read-Methoden

- Für reine Leseabfragen: `AsNoTracking()`
- Erst filtern, dann projezieren
- In Repository nur Datenlogik, keine API-Logik
- Rückgabe für Listen: `IList<T>` oder `List<T>` konsistent halten

Chapter 6. ReadModels / Overview-Objekte

Nutze eigene Typen für Auswertungen statt Entities 1:1 nach außen zu geben.

```
public record EntityOverview(
    int Id,
    string DisplayName,
    bool IsActive,
    int RelatedCount
);
```

6.1. Typisches Query-Template

```
public async Task<IList<EntityOverview>> GetOverviewAsync(string? filter, bool?
onlyActive)
{
    var query = _dbContext.Entities.AsNoTracking().AsQueryable();

    if (!string.IsNullOrWhiteSpace(filter))
        query = query.Where(x => x.Name.Contains(filter));

    if (onlyActive == true)
        query = query.Where(x => x.ActiveUntil == null || x.ActiveUntil >= DateTime
.Today);

    return await query
        .Select(x => new EntityOverview(
            x.Id,
            x.Name,
            x.ActiveUntil == null || x.ActiveUntil >= DateTime.Today,
            x.Children.Count))
        .OrderBy(x => x.DisplayName)
        .ToListAsync();
}
```

Chapter 7. API Design Muster

7.1. DTO statt Entity direkt verwenden

```
public record EntityDto(int Id, string Name, string Type);
```

7.2. Endpoint-Satz (Minimal API)

```
route.MapGet("", async (IUnitOfWork uow) => Results.Ok(await uow.Entities
.GetNoTrackingAsync()));
route.MapGet("/{id:int}", async (int id, IUnitOfWork uow) => { /* by id */ });
route.MapPost("", async (EntityDto dto, IUnitOfWork uow) => { /* create */ });
route.MapPut("/{id:int}", async (int id, EntityDto dto, IUnitOfWork uow) => { /*
update */ });
route.MapDelete("/{id:int}", async (int id, IUnitOfWork uow) => { /* delete */ });
route.MapGet("/overview", async (string? filter, bool? onlyActive, IUnitOfWork uow) =>
{ /* report */ });
```

7.3. API Fehlerbehandlung

- 400 bei ungueltigen Requests
- 404 wenn Ressource fehlt
- 201 bei Create
- 204 bei erfolgreichem Update/Delete ohne Body

Chapter 8. Validierung (FluentValidation)

Halte Regeln im API-Projekt, nicht in Endpoints verstreut.

```
public class EntityDtoValidator : AbstractValidator<EntityDto>
{
    public EntityDtoValidator()
    {
        RuleFor(x => x.Name)
            .NotEmpty()
            .MinimumLength(3)
            .MaximumLength(100);

        RuleFor(x => x.Type)
            .NotEmpty();
    }
}
```

Chapter 9. Import-Pipeline Muster

9.1. Stabiler Ablauf

1. Datei lesen (CSV/JSON)
2. Zeilen validieren
3. Referenzen auflösen (Lookup Tabellen)
4. Neue Datensätze erzeugen oder bestehende aktualisieren
5. In Batches speichern
6. Abschlussbericht ausgeben

9.2. Import Template

```
foreach (var row in importedRows)
{
    var existing = await repository.GetAsync(x => x.ExternalKey == row.ExternalKey);

    if (!existing.Any())
    {
        await repository.AddAsync(new EntityName { ExternalKey = row.ExternalKey, Name
= row.Name });
    }
}

await uow.SaveChangesAsync();
```

Tipps:

- Duplikate über eindeutige Schlüssel erkennen
- Großen Import in mehrere `SaveChangesAsync()` Blöcke teilen
- Fehler pro Zeile sammeln statt sofort abbrechen (optional)